be704ba2-fb77-497e-abfe-bc72f875c49a

<prev_user>

<request>

You are working on an issue in the 'BlackbirdWorks/gopherstack' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gopherstack/gopherstack (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-15T03:52:36.311Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Please start working on issue #{issue_number}: {issue_title}. {issue_url}

Please commit and push often so you don't lose work.

Fixes #689
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

<prev_assistant>

All changes have been committed and pushed. Here's a summary of what was fixed:

Summary of Changes (Issue #689)

`pkgs/lockmetrics/lockmetrics.go` — Metric Leak + Hot-loop Overhead
- **Metric leak fix**: `Close()` now calls `DeletePartialMatch({"lock": name})` on all four metric vectors (`waitSeconds`, `holdSeconds`, `activeWriters`, `activeReaders`) so Prometheus series for the closed mutex are removed. Previously, `Close()` only removed the mutex from the `liveCollector` map, leaving all label-value series permanently allocated.
- **Hot-loop overhead fix**: Pre-curried the `activeReaders` gauge at `New()` time into `activeReadersLock prometheus.Gauge`. `RLock`/`RUnlock` now call `m.activeReadersLock.Inc()/Dec()` directly, eliminating the `WithLabelValues` hash lookup on every read-lock acquisition.

`pkgs/handler/response_writer.go` — Dead Code in `Write()`
- `statusCode` was initialized to `http.StatusOK` (200) in `NewResponseWriter()`, making the `if statusCode == 0` branch in `Write()` permanently unreachable.
- Fixed by initializing `statusCode` to `0`. `Write()` now correctly sets it to 200 when called without a prior `WriteHeader`. `StatusCode()` returns `0` when neither has been called.

`pkgs/events/emitter.go` — Listener Panic Risk + Inefficient Unsubscribe
- **Panic recovery**: Added `ErrListenerPanicked` sentinel error and a `safeCall` generic helper that wraps each listener call in a `recover()`. Panics are returned as `fmt.Errorf("%w: %v", ErrListenerPanicked, r)` from `Emit()`, preventing the entire emit loop from crashing.
- **Efficient unsubscribe**: `removeByID` replaced `append([:i], [i+1:]...)` (O(n) shift + allocation) with swap-with-last (O(1), no allocation). Added `i != last` guard and tail-slot zeroing for GC.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'BlackbirdWorks/gopherstack' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gopherstack/gopherstack (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.

</repository_context>

<current_datetime>2026-03-15T05:18:59.053Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
fix(pkgs): metric leak, dead status-code branch, listener panic crash
</pr_title>

<pr_description>
Three independent bugs in shared packages: Prometheus label-value series leaked on mutex close, a `statusCode == 0` guard that could never fire, and a listener panic that took down the entire `Emit` call.

`pkgs/lockmetrics` — metric leak + per-call label hash overhead
- `Close()` now calls `DeletePartialMatch({"lock": name})` on all four metric vecs, removing every label-value series tied to the closed mutex. Previously only the `liveCollector` map entry was removed, silently leaking series for every dynamic resource (bucket, table, etc.).
- Pre-curry `activeReaders` gauge to lock name at `New()` time (`activeReadersLock prometheus.Gauge`); `RLock`/`RUnlock` now call `Inc()/Dec()` directly instead of re-hashing labels on every acquisition.

`pkgs/handler` — dead branch in `Write()`
`NewResponseWriter` initialized `statusCode` to `200`, making the `if statusCode == 0` guard in `Write()` unreachable. `statusCode` is now zero-initialized; `Write()` sets it to `200` on first call, `StatusCode()` returns `0` when neither `WriteHeader` nor `Write` has been called.

`pkgs/events` — listener panic crashes `Emit`, O(n) unsubscribe
- Added `ErrListenerPanicked` sentinel and a `safeCall` helper that wraps each listener in `recover()`; panics surface as `fmt.Errorf("%w: %v", ErrListenerPanicked, r)` instead of unwinding the caller.

- `removeByID` replaced `append([:i], [i+1:]...)` (O(n) shift + allocation) with swap-with-last (O(1), no allocation), zeroing the vacated tail slot for GC.

```go
// Panic is now contained — Emit returns an error instead of crashing.
emitter.Subscribe(func(_ context.Context, e *MyEvent) error {
    panic("oops")
})
err := emitter.Emit(ctx, event)
errors.Is(err, events.ErrListenerPanicked) // true
```
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>4062133817</comment_id>
<author>@agbishop</author>
copilot-autodev: This PR is addressing Issue #689.
</comment_new>

<comment_new>
<comment_id>4062177962</comment_id>
<author>@agbishop</author>
copilot-autodev: PR requires manual deployment approval for workflow(s): Pull Request CI. Waiting for a reviewer to approve the environment deployment before proceeding.
</comment_new>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Fixes three independent bugs in shared packages: removes leaked Prometheus label series for per-resource locks, makes `ResponseWriter.Write()`'s default-status behavior reachable/accurate, and prevents a panicking event listener from crashing `Emit`.

Changes:
- `pkgs/lockmetrics`: `Close()` now removes all metric series for a lock name; `RLock/RUnlock` use a pre-curried gauge to

avoid per-call label hashing.
- `pkgs/handler`: `ResponseWriter` now zero-initializes status and sets 200 on first `Write()`, making the prior dead branch meaningful.
- `pkgs/events`: `Emit` now recovers listener panics into a sentinel-wrapped error; unsubscribe removal is now O(1).

Reviewed changes

Copilot reviewed 6 out of 6 changed files in this pull request and generated 2 comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>pkgs/lockmetrics/lockmetrics.go:156</file>
<original_commit>9042f53</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The struct layout comment says the scan range arrangement is "minimising pointer bytes to 56", but adding `activeReadersLock prometheus.Gauge` (an interface) changes the pointer word count/GC scan considerations. Please update or remove the hard-coded size claim so it stays accurate as the struct evolves.

</comment_old>

</comment_thread>

<comment_thread>
<file>pkgs/events/emitter_test.go:219-224</file>
<original_commit>9042f53</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Panic recovery is tested via substring matching, but the new

`ErrListenerPanicked` sentinel is intended to be detectable via `errors.Is`/`assert.ErrorIs`. Please add an assertion that the returned error wraps `events.ErrListenerPanicked` in the panic cases so the contract can't regress.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>3949712971</comment_id>
<author>@agbishop</author>
@copilot (refinement check 1 of 2 against issue #689). Please review your implementation against all requirements in the original issue and refine anything that is missing or incomplete. While refining look for any missing features compared to localstack and aws realism (even outside the story spec for extra credit) and any performance issue or optimizations along with any issues with concurrency. Ensure there is no resource leaks. Please address any review feedback.
</comment_new>

</pr_comments>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment

```
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```